CASCADE: Autonomous Orbital Collision Avoidance for Congested LEO

National Space Hackathon 2026 — IIT Delhi

Team B.O.N.K

https://github.com/Aqwerty321/PROJECT-B.O.N.K

March 2026

Abstract

Low Earth Orbit faces an escalating Kessler-syndrome risk as mega-constellation deployments collide with a legacy debris population exceeding 27,000 tracked objects. We present **CASCADE**, a deterministic, single-container Autonomous Constellation Manager that ingests high-frequency ECI telemetry, screens for conjunctions through a multi-tier filtering cascade, and autonomously schedules collision-avoidance and recovery burns under fuel, cooldown, and line-of-sight constraints. The engine processes 50 satellites against 10,000 debris fragments in **13.3 ms** mean tick latency with **zero false negatives** across all test families. A React/Canvas/WebGL frontend delivers a mission-console experience at 60 FPS. CASCADE ships as a single Docker image (`ubuntu:22.04`, port 8000) with 8/8 CTest safety gates enforced in CI, meeting every deployment and API contract specified in the problem statement.

1. Introduction and Design Goals

The National Space Hackathon 2026 challenges teams to build an Autonomous Constellation Manager (ACM) that can (i) ingest high-frequency ECI state vectors, (ii) predict conjunctions up to 24 hours ahead without $O(N^2)$ bottlenecks, (iii) autonomously compute and schedule evasion and recovery maneuvers, and (iv) visualize the operational state in a real-time mission dashboard.

CASCADE addresses these objectives under a strict *safety-first* design philosophy:

- **Zero false negatives**: every ambiguous pair is escalated to a more expensive solver, never silently dropped.
- **Deterministic runtime**: identical inputs always produce the same maneuver sequence, enabling reproducible CI gates.
- **Conservative classification**: the engine prefers false positives (unnecessary screening work) over false negatives (missed collisions).
- **Single-container deployment**: one `docker build` yields a self-contained binary and frontend, exposed on port 8000.

The remainder of this report details the system architecture (§2), numerical methods (§3), frontend console (§4), validation evidence (§5), and deployment pipeline (§6).

2. System Architecture

Figure 1 illustrates the end-to-end data path. The system is a single-process C++20 engine linked against two FetchContent dependencies — `cpp-httplib` v0.15.3 (HTTP) and `simdjson` v3.9.4 (JSON parsing) — with Boost headers for version verification.

```
POST /api/telemetry
  ↓
[State Store] - lock-free SoA upsert
  ↓
[Propagation] - adaptive RK4+J2
  ↓
[Broad Phase] - O(N log N) SMA+incl bins
  ↓
[Narrow Phase] - MOID + TCA + RK4 refine
  ↓
[CDM Engine] - 24 h conjunction horizon
  ↓
[COLA Planner] - RTN evasion + Tsiolkovsky
  ↓
[Recovery] - CW/ZEM slot-targeting burn
  ↓
REST API (port 8000)
```

Figure 1: End-to-end simulation tick pipeline.

2.1 State Representation

All objects are stored in a Structure-of-Arrays (SoA) layout indexed by string identifier. Each record holds:

- 6-DOF ECI state vector $(\vec{r}, \vec{v})$ in km and km/s
- Precomputed Keplerian elements $(a, e, i, \Omega, \omega, M)$
- Radial bounds $(r_{\min}, r_{\max})$ for shell overlap
- Fuel mass, cooldown timer, slot reference, burn history

2.2 API Surface

The engine exposes five primary endpoints on port 8000, bound to `0.0.0.0`:

Endpoint	Purpose
`POST /api/telemetry`	Bulk ECI state ingestion
`POST /api/maneuver/schedule`	Evasion/recovery burn submission
`POST /api/simulate/step`	Advance simulation by Δt s
`GET /api/visualization/snapshot`	Geodetic snapshot for frontend
`GET /api/status`	Engine health & diagnostics

Table 1: Primary REST API endpoints.

Additional debug endpoints (`/api/debug/{burns, conjunctions, propagation, conflicts}`) expose internal ring buffers for observability.

3. Numerical Methods

3.1 Orbital Propagation

The propagator implements an adaptive three-mode scheme:

Mode 1 — J2 Secular Fast Path. For low-eccentricity ($e \leq 0.02$), high-perigee (≥ 500 km) objects with $\Delta t \leq 30$ s, we apply closed-form J2 secular rates to mean elements:

$$\dot{\Omega} = -\frac{3}{2} n J_2 \left(\frac{R_E}{p}\right)^2 \cos i \tag{1}$$

$$\dot{\omega} = \frac{3}{4} n J_2 \left(\frac{R_E}{p}\right)^2 (5\cos^2 i - 1) \tag{2}$$

where $n = \sqrt{\mu/a^3}$, $p = a(1-e^2)$, and $R_E = 6378.137$ km. An extended fast lane admits $e \leq 0.003$, perigee ≥ 650 km, $\Delta t \leq 45$ s. Long horizons are sub-stepped at 300 s intervals.

Mode 2 — Probe-Then-Escalate. Objects outside the fast lane first receive a J2-secular prediction, then a coarse RK4 probe (max step 120 s) validates the result. If drift exceeds 0.5 km in position or 0.5 m/s in velocity, the engine escalates to full RK4.

Mode 3 — Full RK4+J2. Forced for $\Delta t > 21{,}600$ s (6 h), perigee < 350 km, or $e \geq 0.98$. Classical fourth-order Runge–Kutta with maximum substep 60 s, integrating the acceleration:

$$\vec{a}_{J2} = \frac{3}{2} \frac{J_2 \, \mu \, R_E^2}{r^5} \begin{bmatrix} x(5z^2/r^2 - 1) \\ y(5z^2/r^2 - 1) \\ z(5z^2/r^2 - 3) \end{bmatrix} \tag{3}$$

Finite-value guards after every substep prevent NaN propagation.

Validation. Maximum position error at 86,400 s (a full day) is $<$ **0.061 km**, verified by regression gate `phase2_regression_gate`.

3.2 Broad-Phase Candidate Generation

The broad phase reduces the $O(N^2)$ all-pairs space through three sequential filters:

1. **SMA–Inclination Band Indexing.** Objects are binned by semi-major axis ($\Delta a = 500$ km) and inclination ($\Delta i = 20°$). Flat sorted arrays with binary search replace hash maps for cache locality. Neighbor search extends ± 2 bins in each dimension.
2. **Shell Overlap Test.** Perigee $r_{\min} = a(1-e)$ and apogee $r_{\max} = a(1+e)$ are compared with a 50 km margin. Objects with invalid elements fall back to instantaneous radius ± 200 km — an intentional over-approximation that preserves the zero-false-negative guarantee.
3. **D-Criterion Gate.** A simplified Drummond metric:

$$D = \sqrt{(\Delta a/\bar{a})^2 + \Delta e^2 + (2\sin(\Delta i/2))^2} \tag{4}$$

with reject threshold $D > 2.0$. Currently deployed in *shadow mode*: the gate logs hypothetical rejections but does not discard pairs, enabling safe validation before activation.

Fail-Open Path. Objects with invalid elements, $e > 0.2$, or radial span exceeding bin coverage bypass all cheap filters and are checked against every debris object. Detailed fail-open counters are exposed in `/api/status?details=1`.

3.3 Narrow-Phase Collision Screening

Candidate pairs from the broad phase enter a multi-gate narrow pipeline:

1. **Plane/Phase Angular Gate.** Angular momentum vectors computed from (i, Ω). Pairs with orbital plane angle $> 75°$ or mean anomaly phase separation $> 150°$ are rejected. Objects with $e > 0.2$ bypass this gate (fail-open).
2. **MOID Proxy.** A sampled orbit–orbit minimum distance using a 72×72 true-anomaly grid (5,184 evaluations per pair). Reject threshold: 2.0 km. Deployed in shadow mode for safe rollout validation.
3. **Linear TCA Approximation.** Four simultaneous linear-segment minimum-distance estimates (start velocities, end velocities backward, averaged, and secant). The minimum d^2 across all four approximations determines whether the pair passes.
4. **RK4 Micro-Refinement.** Pairs within 0.10 km above the screening threshold receive a fine-grained RK4 propagation ($\Delta t_{\max} = 5$ s).
5. **Full-Window RK4 Refinement.** Pairs within 0.20 km receive a budget-limited, OpenMP-parallelized full-window refinement (16 samples,

1 s substep). Thread-safe atomic budget decrement prevents compute runaway: base budget 64, dynamically scaled by pair count and step size, clamped to $[8, 192]$.

Uncertainty promotion adds 0.10 km to the screening band for pairs with relative speed exceeding 8 km/s. The effective screening distance is:

$$d_{\text{screen}} = \underbrace{0.100}_{\text{collision}} + \underbrace{0.020}_{\text{TCA guard}} = 0.120 \text{ km} \tag{5}$$

3.4 Maneuver Planning

3.4.1 Autonomous COLA

When a predictive CDM record reaches CRITICAL severity ($d < 100$ m), the engine triggers an autonomous evasion burn search over a discrete grid:

- 4 burn-time fractions of the remaining lead time
- 4 ΔV scales: $\{0.5, 1, 2, 4\} \times 0.001$ km/s
- 10 RTN directions: $\pm T$, $\pm R$, 4 diagonal T/R, $\pm N$

The RTN frame is constructed as:

$$\hat{r} = \frac{\vec{r}}{|\vec{r}|}, \quad \hat{n} = \frac{\vec{r} \times \vec{v}}{|\vec{r} \times \vec{v}|}, \quad \hat{t} = \hat{n} \times \hat{r} \tag{6}$$

Candidates are ranked by a multi-criteria comparator: safety class (miss distance relative to target 0.15 km) $\rightarrow$ $|\Delta V|$ $\rightarrow$ normal component magnitude $\rightarrow$ slot error $\rightarrow$ predicted miss distance. The top 3 critical records per satellite are evaluated.

3.4.2 Fuel Accounting

Propellant expenditure follows the Tsiolkovsky relation:

$$\Delta m = m_{\text{current}} \left(1 - e^{-|\Delta \vec{v}|/v_e}\right), \quad v_e = I_{\text{sp}} \cdot g_0 \tag{7}$$

with $I_{\text{sp}} = 300$ s, $g_0 = 9.80665$ m/s^2, $m_{\text{dry}} = 500$ kg, $m_{\text{fuel,init}} = 50$ kg. A 5% fuel guard (2.5 kg) triggers automatic graveyard orbit transfer.

3.4.3 Constraint Enforcement

- **Cooldown**: 600 s mandatory rest between burns on the same satellite.
- **Thrust cap**: $|\Delta V| \leq 15$ m/s per burn.
- **LOS validation**: Upload windows scanned in 20 s steps across a 1,800 s horizon against 6 ground stations. 10 s signal latency enforced.
- **Blackout handling**: Burns must be computed and queued before the satellite leaves the last ground-station footprint.

3.5 Recovery Planning (CW/ZEM)

Every evasion maneuver triggers a paired recovery burn computed via the Clohessy–Wiltshire (CW) state-transition formulation. The default solver (`CW_ZEM_EQUIVALENT`) inverts the in-plane 2×2 Φ_{rv} submatrix:

$$\Delta \vec{v}_{\text{rec}} = \alpha \, \Phi_{rv}^{-1} \, \delta \vec{r} + \beta \, \delta \vec{v} \tag{8}$$

with $\alpha = 0.70$ (position correction weight), $\beta = 0.30$ (velocity damping weight), CW horizon $= 0.25 \times T_{\text{orbit}}$ clamped to $[300, 5400]$ s. The correction is capped at 15% of remaining ΔV budget and 40% of heuristic norm to prevent over-correction. A heuristic fallback (proportional RTN gains) activates when the CW matrix is near-singular ($nt \approx k \cdot 2\pi$).

The station-keeping box is defined as a 10 km spherical radius around the nominal orbital slot. Slot references are bootstrapped from first telemetry and J2-secularly propagated. The regression gate `recovery_slot_gate` enforces $\Delta_{\text{slot}} \leq 0.08$ deterministically.

4. Frontend Mission Console

The frontend is a React 18 / TypeScript single-page application built with Vite and rendered via Canvas 2D and Three.js WebGL. It polls the engine's REST API at 1–2 Hz and presents six operational views:

1. **Command Center** (`#/command`): Fleet-wide posture summary with satellite counts by status, active CDM warnings, recent burn history, and engine health.
2. **Ground Track** (`#/track`): Mercator-projection canvas rendering all satellite markers with 90-minute historical trails, 90-minute forecast paths, live solar terminator overlay, ground station markers, and a persistent map legend with epoch timestamp.
3. **Threat Assessment** (`#/threat`): Polar "bullseye" plot of debris in the selected satellite's proximity. Radial distance encodes time-to-closest-approach; angular position encodes approach vector. Color-coded by CDM severity (green/yellow/red).
4. **Burn Operations** (`#/burn-ops`): Maneuver timeline showing evasion and recovery burns with blackout zone overlays and cooldown friction markers. Fuel gauges per satellite.
5. **Fleet Status** (`#/fleet-status`): Aggregated constellation health with fuel heatmaps, ΔV-cost versus collisions-avoided plots, and station-keeping metrics.
6. **Scorecard** (`#/scorecard`): Judge-facing evidence dashboard rolling up all six PS §7 criteria with weighted readiness percentage, live evidence matrix, and submission gap tracking.

Rendering uses requestAnimationFrame with Canvas 2D for the ground track and Three.js for the 3D polar view. The snapshot endpoint delivers geodetic coordinates and a compressed debris tuple array (`[id, lat, lon, alt]`) to minimize payload size and maintain 60 FPS at 10,000+ debris markers.

5. Validation and Evidence

5.1 Test Infrastructure

CASCADE maintains 8 CTest gate tests, each a standalone C++ binary in `tools/` driven by shell harnesses in `scripts/`. All gates are enforced in GitHub Actions CI, and the Docker build itself fails if any test fails.

Gate	Invariant
`api_contract_gate`	All 5 endpoints conform to PS schema
`broad_phase_validation`	Shell filter $\supseteq$ brute-force baseline
`narrow_phase_false_negative`	Zero missed collisions vs. RK4 reference
`narrow_phase_calibration`	Screening thresholds within tolerance
`phase2_regression`	Propagator drift $<$ 0.061 km at 86,400 s
`maneuver_ops_invariants`	Cooldown, fuel, thrust-cap enforcement
`recovery_planner_invariants`	CW/ZEM convergence, slot error ≤ 0.08
`recovery_slot_gate`	Deterministic slot recovery

Table 2: CTest safety gates and their enforced invariants.

5.2 Safety Score Evidence

- **False negatives**: 0 across 6 test families (broad-phase sanity, narrow-phase canary, false-negative gate, calibration probe, regression, maneuver-ops).
- **Fail-open counters**: Tracked per tick. Every object with uncertain elements is escalated, never discarded.
- **Screening threshold**: Effective 0.120 km with uncertainty promotion to 0.220 km for hypervelocity pairs.

5.3 Fuel Efficiency Evidence

- Tsiolkovsky fuel accounting validated in `maneuver_ops_invariants_gate`.
- CW/ZEM recovery solver minimizes $|\Delta V|$ relative to the heuristic proportional controller.
- Avoidance fuel vs. total fuel tracked separately in burn summaries.

5.4 Algorithmic Speed Evidence

Scenario	Mean Tick	Scale
50 sats / 10,000 debris	13.3 ms	Baseline
100 sats / 20,000 debris	72.6 ms	2× stress

Table 3: Tick latency benchmarks (`phase3_tick_benchmark`).

The broad phase reduces candidate count from $O(N \cdot M)$ to a small fraction via SMA/inclination binning. OpenMP parallelization of the narrow-phase sweep yields a 31.5% speedup at 2× scale.

5.5 Deployment Verification

The Docker API smoke gate (`scripts/docker_api_smoke.sh`) performs an automated release-verification cycle:

1. Build Docker image from `Dockerfile`
2. Boot container on auto-selected port
3. Probe `/api/status`, POST telemetry, POST simulate/step, GET snapshot
4. Validate each response with Python assertions
5. Tear down container

This gate runs in CI and catches regressions before any push reaches main.

6. Deployment

6.1 Docker Build

CASCADE uses a 3-stage multi-stage Dockerfile:

Stage	Base	Purpose
builder	`ubuntu:22.04`	GCC-12, CMake, CTest gates
frontend	`node:24-slim`	`npm ci` + Vite production build
runtime	`ubuntu:22.04`	Minimal: `libgomp1` only

Table 4: Docker build stages.

BuildKit cache mounts accelerate rebuilds (`apt`, `_deps`, `sccache`). A `PROJECTBONK_PREFETCH_ONLY` step prefetches all FetchContent dependencies before source copy, maximizing layer-cache hits. The runtime image runs as a non-root user (`bonk`).

6.2 Quick Start

```
# Build and run
docker build -t cascade:local .
docker run --rm -p 8000:8000 cascade:local

# Smoke test
curl -s http://localhost:8000/api/status \
  | python3 -m json.tool

# Full verification
./scripts/docker_api_smoke.sh
./scripts/run_ready_demo.sh
```

6.3 CI Pipeline

GitHub Actions runs on every push to `main`:

1. CMake configure + build (`gcc-12`, Release)
2. CTest: all 8 safety gates
3. Frontend: `npm ci && npm run build`
4. Docker API smoke gate

7. Limitations and Future Work

- **Propagation fidelity**: The engine uses J2-only perturbation. Atmospheric drag is applied heuristically for perigee $<$ 350 km orbits. Lunar/solar third-body and SRP effects are not modeled.
- **MOID solver**: The current 72×72 sampled proxy achieves sufficient accuracy for hackathon-scale screening but could be replaced with a Gronchi (2005) analytical quartic solver for production use.
- **Maneuver optimization**: The grid-search COLA planner explores a discrete candidate space. A continuous NLP or sequential convex programming approach would yield lower-ΔV evasion sequences.

- **Dense-scene FPS**: Frontend rendering at 10,000+ debris has been tested in development but formal 60 FPS proof under worst-case load needs finalization.
- **Demo video**: The narrated demonstration video is planned around `scripts/run_ready_demo.sh` but awaits final recording.

Acknowledgments

We thank the organizers of the National Space Hackathon 2026 at IIT Delhi for designing a challenging and technically rigorous problem statement.

A. Key Physical Constants

Symbol	Description	Value
μ	Earth grav. parameter	$398\,600.4418\,\mathrm{km}^3/\mathrm{s}^2$
R_E	Earth equatorial radius	$6\,378.137\,\mathrm{km}$
J_2	Oblateness coefficient	1.08263×10^{-3}
d_{coll}	Collision threshold	$0.100\,\mathrm{km}$
I_{sp}	Specific impulse	$300\,\mathrm{s}$
g_0	Standard gravity	$9.80665\,\mathrm{m/s}^2$
m_{dry}	Satellite dry mass	$500\,\mathrm{kg}$
m_{fuel}	Initial propellant	$50\,\mathrm{kg}$
$\lvert\Delta V\rvert_{\max}$	Thrust limit	$15\,\mathrm{m/s}$
τ_{cd}	Cooldown period	$600\,\mathrm{s}$
m_{EOL}	Fuel EOL guard	$2.5\,\mathrm{kg}$
τ_{sig}	Signal latency	$10\,\mathrm{s}$
r_{box}	Station-keeping box	$10\,\mathrm{km}$

Table 5: System constants (from `src/types.hpp`).

B. CDM Severity Tiers

Tier	Condition	Auto-COLA
CRITICAL	$d < 0.100\,\mathrm{km}$	Yes
WARNING	$d < 1.0\,\mathrm{km}$	No (monitor)
WATCH	$d < 5.0\,\mathrm{km}$	No (log)

Table 6: Conjunction severity classification.

C. Verification Commands

```
# Build (local)
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --target ProjectBONK

# Run all safety gates
ctest --test-dir build --output-on-failure

# Individual gates
./scripts/api_contract_gate.sh ./build
./scripts/broad_phase_sanity_gate.sh
./scripts/narrow_phase_false_negative_gate.sh
./scripts/recovery_planner_invariants_gate.sh

# Docker release verification
./scripts/docker_api_smoke.sh

# Frontend build
cd frontend && npm ci && npm run build

# Ready demo
./scripts/run_ready_demo.sh
```

D. NSGA-II Tuner Parameters

The offline multi-objective tuner optimizes 69 parameters across 3 tiers (broad phase, narrow phase, maneuver planner) against 3 objectives: safety (zero false negatives), fuel efficiency (total ΔV), and speed (tick latency). The tuner runs as a standalone C++ Monte-Carlo harness, separate from the production runtime. Parameter values are injected at compile time or via environment variable overrides (all 69 constants are `PROJECTBONK_*` configurable).